Credit Name: Chapter 13
Assignment Name: ReverseList
Noorinder

How has your program changed from planning to coding to now? Please explain.

Initially, the program was designed to reverse a series of integers entered by the user using a stack. The implementation largely followed the original plan, but certain challenges and improvements emerged during the coding process to ensure the program worked as expected. Below are the key issues encountered and how they were addressed.

1. Missing Stack Initialization
Problem:
The stack was not initialized before being used to store integers, which caused a `NullPointerException` when attempting to push elements.

Fix:
I added code to initialize the stack before starting to collect user inputs.

Code Example:

```
// Initialize the stack before use
   Stack<Integer> stack = new Stack<>();
```

2. Lack of Input Validation
Problem:
The program did not validate user input, allowing non-integer or invalid entries to cause runtime exceptions.

Fix:
I added a check to ensure that the user enters only integers. Invalid inputs prompt the user to try again.

Code Example:

```
while (stack.size() < 10) {
    System.out.println("Enter a number (999 to quit): ");
    if (input.hasNextInt()) { // Check if the input is an integer
        int num = input.nextInt();
        if (num == 999) {
            break;
        }
        stack.push(num); // Push valid numbers onto the stack
    } else {
        System.out.println("Invalid input. Please enter an integer.");
        input.next(); // Clear the invalid input
    }
}
```

3. Exiting Without Proper Notification
Problem:
The program exited the input loop without notifying the user when the stack reached the maximum size or the user decided to quit.

Fix:
I added a message to inform the user when the program stops collecting inputs, either because the stack is full or the user entered 999.

Code Example:

```
if (stack.size() == 10) {
    System.out.println("Maximum of 10 numbers reached. Stopping input.");
} else if (num == 999) {
    System.out.println("Input stopped by user.");
}
```

4. Inefficient Reverse Output
Problem:
The program directly printed numbers in reverse without storing them for further operations or formatting.

Fix:
I modified the program to collect reversed numbers into a list for better formatting or additional operations before output.

Code Example:

```
System.out.print("Numbers in reverse: ");
List<Integer> reversedNumbers = new ArrayList<>();
while (!stack.isEmpty()) {
    reversedNumbers.add(stack.pop()); // Store numbers in a list
}
System.out.println(reversedNumbers); // Print the list in reverse order
```

5. Resource Management
Problem:
The `Scanner` object was not closed in earlier versions, leading to potential resource leaks.

Fix:
I explicitly closed the `Scanner` at the end of the program to release resources.

Code Example:

```
input.close(); // Close the Scanner to release resources
```

Final Thoughts:
These changes enhanced the program's functionality, efficiency, and user experience. The added input validation and user notifications made the program more robust, while the structured reverse output improved its versatility. The final version of the program works seamlessly to reverse a series of integers entered by the user.